

Puzzle con DFS Recursivo

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. DFS Recursivo	2
2.1. Concepto	2
2.2. Implementación	2
2.3. Ventajas y desventajas	2
3. BFS para Comparación	2
4. Animación de la Solución	3
5. Interfaz Gráfica	3

1. Descripción del Proyecto

Visualizador comparativo de dos algoritmos de búsqueda aplicados a un puzzle: **DFS Recursivo** y **BFS**. Permite ver animadamente cómo cada algoritmo explora el espacio de estados hasta encontrar la solución.

2. DFS Recursivo

2.1. Concepto

El DFS Recursivo explora tan profundo como sea posible por cada rama antes de retroceder (backtrack). Su implementación recursiva delega el manejo de la pila al sistema operativo.

2.2. Implementación

```
1  Nodo_DFS resolverDFS(Nodo_DFS nodo ,
2                          List<Nodo_DFS> visitados) {
3      if (esMeta(nodo.getEstado())) return nodo;
4
5      for (int[] estadoHijo : generarHijos(
6                          nodo.getEstado())) {
7          if (!estaEnLista(visitados, estadoHijo)) {
8              Nodo_DFS hijo = new Nodo_DFS(estadoHijo);
9              hijo.setPadre(nodo);
10             visitados.add(hijo);
11
12             Nodo_DFS resultado =
13                 resolverDFS(hijo, visitados);
14             if (resultado != null) return resultado;
15             // si retorna null, hace backtrack
16         }
17     }
18     return null; // no encontró solución por esta rama
19 }
```

2.3. Ventajas y desventajas

- **Ventaja:** código simple y elegante, menos memoria que BFS.
- **Desventaja:** no garantiza solución óptima; riesgo de `StackOverflowError` en problemas profundos.

3. BFS para Comparación

Se implementa BFS en la misma GUI para comparar visualmente cuántos pasos tarda cada algoritmo:

```
1  Nodo_BFS resolverBFS(int[] estadoInicial) {
```

```

2 Queue<Nodo_BFS> cola = new LinkedList<>();
3 List<Nodo_BFS> visitados = new ArrayList<>();
4 Nodo_BFS raiz = new Nodo_BFS(estadoInicial);
5 cola.add(raiz);
6 visitados.add(raiz);
7
8 while (!cola.isEmpty()) {
9     Nodo_BFS actual = cola.poll();
10    if (esMeta(actual.getEstado())) return actual;
11
12    for (int[] hijo : generarHijos(
13        actual.getEstado())) {
14        if (!estaVisitado(visitados, hijo)) {
15            Nodo_BFS nHijo = new Nodo_BFS(hijo);
16            nHijo.setPadre(actual);
17            visitados.add(nHijo);
18            cola.add(nHijo);
19        }
20    }
21 }
22 return null;
23 }

```

4. Animación de la Solución

Una vez encontrada la solución, se reconstruye el camino y se anima:

```

1 void animarSolucion(List<int[]> camino) {
2     int[] paso = {0};
3     Timer timer = new Timer(500, null);
4     timer.addActionListener(e -> {
5         if (paso[0] < camino.size()) {
6             estadoActual = camino.get(paso[0]);
7             repaint();
8             paso[0]++;
9         } else {
10             ((Timer)e.getSource()).stop();
11         }
12     });
13     timer.start();
14 }

```

5. Interfaz Gráfica

- Visualización del puzzle con fichas numeradas.
- Botón **Auto DFS**: ejecuta y anima la solución con DFS recursivo.
- Botón **Auto BFS**: ejecuta y anima la solución con BFS.

- Contador de pasos para comparar ambos algoritmos.